

BDML - Abschlussprojekt - WiSe 25/26

Benjamin Fresz

Alexander Baust

18. Dezember 2025

Infos zum Datensatz

Reinforcement Learning in der Kabelmontage

Die Kabelmontage umfasst das Verlegen, Führen und Befestigen von elektrischen Leitungen sowie das Herstellen sicherer Steckverbindungen. Hierfür muss ein Stecker in eine Buchse geführt werden (=Konnektorinstallation). Diese Aufgabe erfordert hohe Genauigkeit bei der Ausrichtung sowie feinfühliges Kraftregelung, um Fehlstellungen oder Beschädigungen zu vermeiden. Zusätzlich erschwert das biegsame Verhalten des Kabels die Automatisierung.

Angesichts dieser Herausforderungen stößt klassische, ablaufbasierte Robotik bei der automatisierten Kabelmontage schnell an ihre Grenzen. **Reinforcement Learning (RL)** hingegen kann durch Interaktion mit der Umgebung (hier: der Roboter, das Kabel und die Buchse) eigenständig Strategien entwickeln, um den Stecker auch unter schwierigen Bedingungen zuverlässig einzustecken.

RL ist ein Teilbereich des maschinellen Lernens, bei dem ein **Agent** lernt, in einer **Umgebung** durch **Ausprobieren** eine **Strategie (Policy)** zu entwickeln, um eine möglichst hohe **Belohnung** zu erhalten.

RL basiert auf folgenden Elementen:

- **Zustand** ($s \in \mathcal{S}$): die aktuelle Situation der Umgebung, z. B.: Position des Steckers.
- **Aktion** ($a \in \mathcal{A}$): Handlung des Agenten, z. B.: Drehen des Roboterarmes.
- **Reward** ($r \in \mathbb{R}$): Belohnung der Umgebung nach einer Aktion.
- **Policy** ($\pi(a|s)$): Strategie, mit der der Agent Aktionen auswählt.
- **Value Function** ($V^\pi(s)$): Erwartete diskontierte Summe zukünftiger Belohnungen ab Zustand s .

Der Agent soll eine Policy π^* lernen, die die Value Function maximiert:

$$\pi^* = \arg \max_{\pi} V^\pi(s) = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

wobei $\gamma \in [0, 1]$ ein Abzinsungsfaktor / Discountfaktor ist, der dazu führt, dass erwartete Rewards in fernerer Zukunft geringer gewichtet werden als in näherer Zukunft.

Beim automatisierten Montieren von Kabeln mit Robotern stößt Reinforcement Learning (RL) auf mehrere Herausforderungen:

- **Hohe Dimensionalität:** Die Zustände (z. B. Kabelpositionen, Roboterpose) und Aktionen (z. B. kontinuierliche Bewegungen) liegen in hochdimensionalen Räumen.
- **Deformierbare Objekte:** Kabel sind flexibel und verformen sich unvorhersehbar, was die Modellierung der Umgebung erschwert.
- **Sparse Rewards:** Belohnungen treten erst am Ende einer erfolgreichen Montage auf.
- **Sicherheitsanforderungen:** Fehlerhafte Aktionen können zu Schäden am Roboter oder Werkstück führen. Daher ist das Ausprobieren in der echten Welt riskant.

Pretraining eines RL-Agenten

Expertendemonstrationen können zum Vortrainieren (Pretraining) eines Agenten genutzt werden. Dabei werden Kabelmontagen eines menschlichen Experten aufgezeichnet, um daraus eine initiale Policy π_0 mittels Imitation Learning (z.B. Behavioral Cloning) zu lernen. Diese Policy kann nun auf den Roboter übertragen werden und erlaubt dem Agenten bereits die Steuerung des Roboters und eine erste autonome Ausführung der Aufgabe. Dadurch wird anschließend der eigentliche Lernprozess beschleunigt, fehlerhafte Exploration in der realen Welt reduziert und die Sicherheit des Trainingsprozesses erhöht.

Versuchsaufbau

Zur Umsetzung dieser Idee wurde eine Roboterzelle inklusive Roboter und Autotür aufgebaut, siehe Abbildung 1. Der Roboter soll den Stecker mit der Buchse in der Autotür verbinden. Für das Aufzeichnen der Expertendemonstrationen wird der Roboter mit einer 3D-Spacemouse, siehe Abbildung 2, gesteuert. Die Daten werden in Episoden aufgezeichnet. Jede Episode startet mit der zufälligen Initialisierung der Position und Orientierung des Roboterarmes innerhalb des Suchbereiches und endet mit dem erfolgreichen Einstecken des Steckers oder mit dem Abbruch der Episode. Abbruchkriterien sind eine Zeitüberschreitung oder ein Verlassen des Suchbereiches. Dieser ist durch eine Verschiebung von maximal $\pm 3,5$ cm und eine Verdrehung des Roboterarmes von maximal $\pm 30^\circ$ relativ zur Buchse eingeschränkt.

Daten

Die gesammelten Daten je Episode werden in CSV-Dateien mit den Namen `episode#.csv` gespeichert. Sie beinhalten die Messwerte des Zustandes, die Aktionen des menschlichen Demonstrators sowie allgemeine Informationen gemäß Tabelle 1.

Die Kräfte und Momente (=Wrench), sowie die (Winkel-)Geschwindigkeiten (=Twist) sind jeweils an der Aufnahme für den Greifer des Roboters (Fachbegriff: TCP – Tool Center Point) definiert. Der Reward r_t zum Zeitschritt t ist mit der folgenden Formel definiert:

$$r_t = \begin{cases} 1, & \text{wenn der Stecker erfolgreich in die Buchse eingesteckt wurde} \\ 0, & \text{ansonsten} \end{cases}$$

Insgesamt wurden zwei Datensätze aufgezeichnet, in dem ersten werden unterschiedliche Strategien für das Einstecken untersucht und der zweite dient dem Training der Modelle. Das Layout der Daten sowie der finalen Skripte ist in Abbildung 3 dargestellt.

Koordinatensystem des Roboters

Es wird ein rechtshändiges Koordinatensystem verwendet, dessen Ursprung sich in der Basis des Roboters befindet. Die Definition der Richtungen der Achsen kann der Abbildung 4 entnommen werden. Der Mittelpunkt der Buchse liegt bei $y = 0.14$ und $z = 0.55$ Meter.

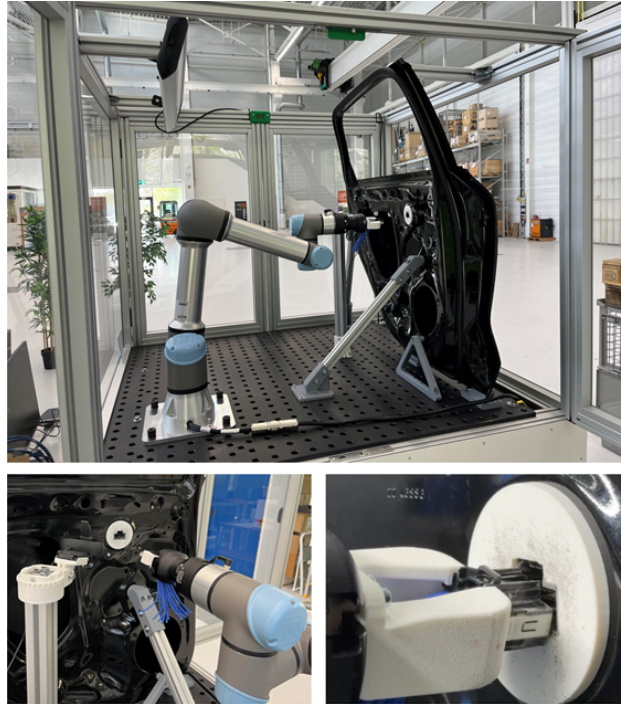


Abbildung 1: Aufbau der Roboterzelle (**oben**), Roboter und Autotür (**unten links**), Position der Kamera (**unten rechts**). Der Greifer des Roboters hält den Stecker und führt diesen in die Buchse.



Abbildung 2: 3D-Spacemouse zur Steuerung des Roboters

Hinweis: Die Endposition einer erfolgreichen Episode wird aufgrund von Messrauschen und -ungenauigkeiten nicht exakt mit dem Mittelpunkt der Buchse übereinstimmen.

Kategorie	Variable	Beschreibung	Einheit
Allgemein	step	Aktueller Schritt t der Episode	–
	ros_timestamp	Zeit seit dem 1. Januar 1970	s
Zustand / State	ft_wrench_fx, fy, fz	Kräfte in x-, y- und z-Richtung	N
	ft_wrench_tx, ty, tz	Momente um x-, y- und z-Achse	Nm
	current_absolute_pose_x, y, z	Aktuelle Position des Steckers	m
	current_twist_vx, vy, vz	Geschwindigkeit	m/s
	current_twist_wx, wy, wz	Winkelgeschwindigkeit	rad/s
Aktionen	action_f_x, y, z	Sollkraft	N
	action_t_x, y, z	Sollmomente	Nm
Reward	reward	Reward zum Zeitschritt t	–

Tabelle 1: Übersicht der Spalten in den Dateien `episode_#.csv`

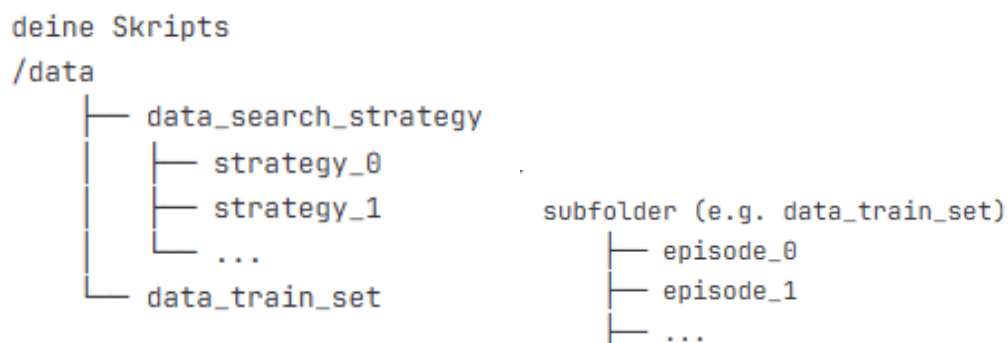


Abbildung 3: Daten-Layout: (**links**) Ordnerstruktur (**rechts**) Struktur eines Ordners.

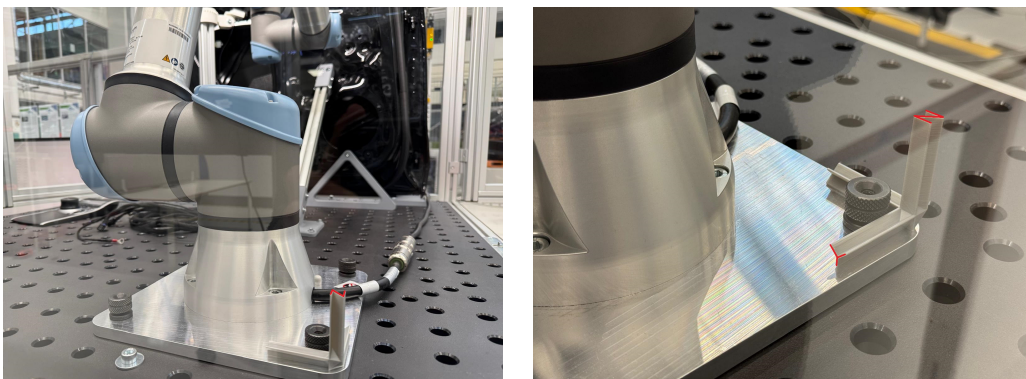


Abbildung 4: Koordinatensystem des Roboters. Die z-Achse zeigt nach oben und die x-Achse zeigt in Richtung der Autotür.

Aufgaben

Alle Skripte und Notebooks müssen kommentiert werden, um die Arbeitsschritte nachvollziehen zu können. Alle Plots müssen mit Titel, Achsenbeschriftung und Legende versehen sein. Insgesamt können 100 Punkte erreicht werden.

1 Maschinelles Lernen mit dem California Housing Datensatz und sklearn (33 Punkte)

In Aufgabe 1 sollen verschiedene Verfahren des Maschinellen Lernens auf dem California Housing Datensatz angewendet werden. Dieser beinhaltet Daten zur Prädiktion des mittleren Hauspreises in einem sogenannten “Housing Block”, also einer Gruppe von Häusern in Kalifornien.

Speichere das Skript unter dem Namen “housing_vorname_nachname.ipynb” (dabei soll selbstverständlich der eigene Name eingesetzt werden).

1.1 Entscheidungsbaum (10 Punkte)

1. Lade den California Housing Datensatz mit sklearn mittels `sklearn.datasets.fetch_california_housing()`.
2. Teile den Datensatz in einen Trainings- und Testdatensatz auf (80% Training, 20% Test).
3. Um welche Art maschineller Lernaufgabe handelt es sich hier? Nenne fünf mögliche ML-Verfahren, mit denen dieses Problem gelöst werden könnte.
4. Wie funktioniert ein Entscheidungsbaum für diese Art Aufgabe?
5. Trainiere einen Entscheidungsbaum auf den Features des Datensatzes zur Prädiktion des Hauspreises für einen Housing Block.
6. Berechne den Mean Squared Error (MSE) des Modells auf den Testdaten.
7. Visualisiere die Verteilungen der Vorhersagen des Entscheidungsbaumes und die der echten Werte in einem 2D-Plot. Verwende hierfür `MedInc` als x-Achse und die Prädiktion (Hauspreis) als y-Achse. Was fällt auf?

1.2 Random Forest Regressor (12 Punkte)

Ein Random Forest besteht aus einer Menge von Entscheidungsbäumen, die auf zufälligen Teilmengen der Features im Datensatz trainiert werden. Jeder Baum innerhalb eines Random Forest gibt eine Vorhersage ab, und die endgültige Vorhersage wird durch Mehrheitsabstimmung (Klassifikation) oder Durchschnitt der Einzelvorhersagen (Regression) getroffen.

1. Erkläre die Vor- und Nachteile eines Random Forest Regressors im Vergleich zu einem einzelnen Entscheidungsbaum.
2. Trainiere einen Random Forest Regressor mit `sklearn`. Verwende alle verfügbaren Features des Datensatzes und teile die Daten erneut in Trainings- und Testdatensatz auf.
3. Berechne den MSE und den R^2 -Score des Modells auf den Trainings- und Testdaten.
4. Führe ein Hyperparameter-Training durch, indem du mindestens 6 Kombinationen der Hyperparameter `n_estimators` und `max_depth` testest. Verwende dafür `GridSearchCV` von `sklearn`.
5. Gib die besten Hyperparameter und den MSE der Trainings- und Testdaten für diese Kombination aus.
6. Beim Training eines Random Forest werden die einzelnen Entscheidungsbäume nicht auf allen Datenpunkten (auch Bootstrapping genannt) und nicht mit allen Features (durch die Einstellung von `max_features` in `sklearn.RandomForestRegressor`) trainiert. Warum ist dieses Vorgehen sinnvoll? Welche Auswirkungen hat es auf den Bias und die Varianz des finalen Modells im Vergleich zu einem Modell, bei dem diese Techniken nicht angewendet werden?

1.3 Neuronales Netzwerk (11 Punkte)

1. Teile den Datensatz in drei Teile: Trainingsdaten (70%), Validierungsdaten und Testdaten (jeweils 15%).
2. Erstelle ein kleines neuronales Netzwerk mit `Keras` zur Vorhersage von Hauspreisen basierend auf den Features des California Housing Datensatzes. Dabei soll das neuronale Netz eine verdeckte Schicht (hidden layer) haben.
3. Implementiere eine Hyperparameter-Optimierung für das neuronale Netzwerk (ohne `sklearn`), um die besten Hyperparameter für die Anzahl der Neuronen in der verdeckten Schicht und die Learning Rate zu finden. Hier sollen wieder 6 verschiedene Hyperparameter-Kombinationen getestet werden. Verwende die Validierungsdaten, um die Ergebnisse der jeweiligen Trainingsläufe zu prüfen. Trainiere die Netzwerke jeweils mit 100 Episoden und einer `batch_size` von 32.
4. Erkläre die Bedeutung der Anzahl der Neuronen in der verdeckten Schicht und der Learning Rate.
5. Gebe die besten Hyperparameter und den MAE des Modells auf den Trainings-, Test- und Validierungsdaten aus.

2 Suchstrategien (24 Punkte)

Hinweis: Verwende für die folgende Aufgabe nur den Datensatz in dem Ordner “data_search_strategy”.

Hinweis: Speichere das Skript unter: “suchstrategie_vorname_nachname.ipynb”

2.1 Datensichtung (8 Punkte)

1. Lies mit Hilfe von Pandas die Episode 0 der Suchstrategie Nummer 2 ein.
2. Wie hoch ist die durchschnittliche Abtastzeit je Step in Millisekunden? Wie viele Schritte waren in der Demonstration nötig, den Konnektor zu installieren? Wie lange hat die Installation in Millisekunden gedauert?
3. Enthalten die Daten fehlende Werte? Wenn ja, wie viele? Welche Spalten enthalten fehlende Werte?
4. Fülle die fehlenden Werte auf. Wähle hierbei eine für Zeitreihendaten geeignete Methode und erkläre, wieso diese gegenüber anderen Alternativen gewählt wurde.
5. Berechne den Value $V^\pi(s)$ der Episode. Verwende hierbei einen Discount-Faktor γ von 0.999. (**Hinweis:** siehe Erklärungen zu RL im ersten Abschnitt)
6. Erkläre, wie trotz der gegebenen Rewardfunktion erzielt werden kann, dass schnellere Einsteckvorgänge präferiert werden.

2.2 Visualisierung der Daten (16 Punkte)

1. Erzeuge für alle drei Koordinatenachsen Histogramm-Plots der Kräfte sowie der Momente. Die jeweiligen Soll- und Ist-Werte sollen dabei nebeneinander (links: Ist, rechts: Soll) in einer gemeinsamen Plot-Figur dargestellt werden. Verwende jeweils Achsenbeschriftungen. Eine Legende ist nicht notwendig, aber jeder Subplot soll einen passenden Titel enthalten sowie die gesamte Figur einen übergeordneten Titel. Verwende je Plot die gleiche Skala für die y-Achse, um die Unterschiede zwischen den Ist- und Sollwerten besser zu verdeutlichen. Verwende für die Histogramme eine Anzahl von 50 Bins. Die Plots sollen in einer gemeinsamen Figur dargestellt werden.
2. Was fällt dir auf? Was sind mögliche Ursachen für die Abweichungen der Verteilungen zwischen Ist- und Sollwerten?
3. Plote den Einsteckvorgang in der Ebene der Autotür (**Hinweis:** siehe Abschnitt zum verwendeten Koordinatensystem). Hebe hierbei die erste und die letzte Position farblich hervor. Der Start soll als roter und das Ende als grüner Punkt dargestellt werden. Ergänze zudem die Position des Mittelpunktes der Buchse. Ergänze einen Titel, eine Legende und eine Achsenbeschriftung. Plote die Suchstrategie relativ zur Position der Buchse.

4. Beschreibe die Suchstrategie des menschlichen Demonstrators: Wie wurde versucht, das Kabel in die Buchse zu stecken?
5. Lies nun auch die restlichen Episoden ein, fülle jeweils fehlende Daten mit der gleichen Methode wie oben auf.
6. Plote die einzelnen Suchstrategien. Erzeuge hierfür einen gemeinsamen Plot mit 4 Zeilen. Jede Zeile soll die Darstellung einer Suchstrategie enthalten. Sortiere diese absteigend nach dem Value der jeweiligen Episode. Verwende hierbei erneut den obigen Diskontfaktor. In der Spalte soll die Suchstrategie wie in Teilaufgabe 3 dargestellt werden. (Idealfall: Automatische Sortierung)
7. Beschreibe die restlichen Suchstrategien. Was fällt dir auf?
8. Plote die Suchstrategien als 3D-Plots über die Position in x, y und z-Richtung.

3 Datenvorverarbeitung (16 Punkte)

Hinweis: Verwende für die folgende Aufgabe nur den Datensatz in dem Ordner “data_train_set”. Neben den absoluten Positionen wurden hier auch die Orientierungen des Steckers aufgezeichnet. Diese sind als Quaternionen dargestellt. Die jeweiligen Spalten sind wie folgt benannt: “current_absolute_pose_q{x; y; z; w}”.

Hinweis: Speichere das Skript unter: “datenvorverarbeitung_vorname_nachname.ipynb”

1. Wie viele Episoden wurden aufgezeichnet? Wie viele Datenpunkte gibt es insgesamt?
2. Lese alle Episoden ein. Es war nicht jede Episode erfolgreich, allerdings soll nur auf erfolgreichen Episoden trainiert werden. Woran kann erkannt werden ob die Episode erfolgreich war? Welcher Prozentsatz der Episoden war erfolgreich? Filtere alle nicht erfolgreichen Episoden aus dem Datensatz.
3. Fülle die Daten wie in Aufgabe 2 auf.
4. Welche Suchstrategie wurde hier verwendet? (**Hinweis:** Für jede Episode wurde die gleiche Suchstrategie verwendet)
5. Berechne für jede verbleibende Episode den Value für jeden Step. Nenne diese Spalte “value”. Verwende hierbei einen Discount-Faktor γ von 0.999.
6. Berechne den Mittelwert der Values zu Beginn der Episode und plote diese Values als Boxplot.
7. Filtere die Ausreißer, die in dem Boxplot angezeigt werden. Wie sind Ausreißer in Boxplots in matplotlib.pyplot (plt) definiert? Wie viele Ausreißer gibt es?
8. Erstelle ein Histogramm der Episodenlängen. Verwende 50 Bins. Wie viele Episoden sind kürzer als 60 Schritte und wie viele länger als 120 Schritte? Wie lange sind die kürzesten und längsten Episoden?

9. Füge alle verbliebenen Episoden zu einem großen Datensatz zusammen.
10. Speichere die Daten für ein späteres Training als CSV-Datei ab. Benenne die Datei "dataset". Dieser Datensatz soll am Ende nicht mitgeschickt werden!

4 Behavioural Cloning (27 Punkte)

In dieser Aufgabe soll ein neuronales Netz mittels Behavioural Cloning trainiert werden, Verwende den zuvor abgespeicherten Datensatz "dataset". Speichere das Skript unter folgendem Namen: "behavioural_cloning_vorname_nachname.ipynb"

1. Setze alle für Keras notwendigen Seeds auf 42. Wieso ist das sinnvoll?
2. Teile den Datensatz in einen Trainings- und Testdatensatz auf. Verwende 75% der Daten für den Trainingsdatensatz. Verwende die aktuelle Positionen sowie Orientierung, den Wrench- und Twistvektor als Inputfeatures und die Aktionen des menschlichen Demonstrators als Labels.
3. Skaliere die Trainings- und Testdaten mit einem MinMaxScaler.
4. Erkläre warum beim Training vieler ML-Modelle die Skalierung der Daten notwendig ist. Nenne 2 Gründe. Nenne ein ML-Verfahren, bei dem keine Skalierung durchgeführt werden muss. Wieso ist hier keine Skalierung notwendig?
5. Trainiere ein neuronales Netz mit Hilfe von Keras (nicht sklearn!). Verwende den MSE Loss. Berechne und printe den Loss auf den Test-Daten. Erreiche mindestens einen Test-Loss von unter 0.01. Verwende mindestens einmal Dropout beim Training und verwende Kreuzvalidierung im Training.
6. Erkläre die Bedeutung von Regularisierung sowie das verwendete Verfahren für die Regularisierung.
7. Plote den Loss und den Validierungs-Loss im Training über die Epochen. Ergänze zudem den Test Loss als horizontalen Strich. Erkläre die Bedeutung der Plots. Was fällt dir auf? Was ist der Grund für das Verhalten?
8. Visualisiere die Vorhersagen des Modells. Lies hierfür eine beliebige Episode aus dem Datensatz ein (die nicht gefiltert wurde) und plote die Vorhersagen des Modells gegen die Aktionen des Demonstrators. Interpretiere die Ergebnisse.
9. Welche Probleme können auftreten, wenn ein mit Behavioural Cloning trainiertes Modell vom Trainingsdatensatz auf das reale System übertragen wird? Nenne drei Probleme, die auftreten könne.
10. Es soll ein neuer Demonstrationsdatensatz für Behavioural Cloning aufgezeichnet werden. Formuliere drei Maßnahmen, die über die reine Datenmenge hinausgehen und schon bei der Aufzeichnung berücksichtigt werden können, damit das trainierte Modell robust arbeitet und erfolgreich auf den Roboter übertragen werden kann.

Hinweise

Falls die Vorverarbeitung aller Daten rechnerisch zu aufwendig für deinen Computer ist, können alle Aufgaben auch mit weniger Messungen / Episoden durchgeführt werden. Bespreche dies aber mit den Betreuern. Allgemein empfiehlt es sich, alle Aufgaben und Vorverarbeitungsschritte zunächst nur an einem kleinen Teil des gesamten Datensatzes durchzuführen, um die korrekte Funktionalität zu testen, bevor der gesamte Datensatz genutzt wird.

Für das Einlesen der Daten sollen relative Pfade anstatt absoluter Pfade verwendet werden, damit wir den Code nach der Abgabe ausführen können.

Nicht lauffähiger Code, zum Beispiel Syntaxfehler oder absolute Pfade, führt in der jeweiligen Aufgabe zu Punktabzug. Verwende in allen Plots Achsenbeschriftungen, eine Legende und einen Titel.

Für das Abschlussprojekt gilt strikte Einzelarbeit. Jede Leistung ist eigenständig zu erbringen. Gruppenarbeit, gemeinsame Code-Erstellung, Austausch oder Wiederverwendung von Lösungen anderer sind untersagt.

Die Nutzung von KI-Tools (z.B. ChatGPT, Copilot, Gemini, KI-Text- oder Codegeneratoren, KI-Fehlersuche oder -Optimierung) ist für dieses Projekt nicht erlaubt. Erlaubt sind nur offizielle Dokumentationen und Lehrmaterialien der Veranstaltung.

Verstöße gegen diese Regeln gelten als Täuschungsversuch und können zur Bewertung mit „nicht bestanden“ sowie zu weiteren Maßnahmen nach Prüfungsordnung führen.

Abgabe

Die vier Notebooks/Skripte (je eines pro Aufgabe) müssen allesamt in einem Ordner abgelegt werden, der nach dem Studierenden benannt ist (Name: “bdml_abgabe_vorname_nachname”). Die gespeicherten und generierten Daten müssen vorher entfernt werden. Es empfiehlt sich vor der Abgabe, den Code-Editor nochmals zu schließen und anschließend alle Skripte nochmals auszuführen. Bis zum 02.März 2025, 23:59 Uhr MEZ sollen die Skripte als .zip-Datei an benjamin.fresz@ipa.fraunhofer.de und alexander.baust@ipa.fraunhofer.de versendet werden. Wir versenden danach so schnell wie möglich eine Empfangsbestätigung.

Falls es Probleme mit Paketen, Python-Installationen oder sonstige Unklarheiten gibt, wende dich bitte frühzeitig per Mail an uns.

Benotung

Das Abschlussprojekt wird für jeden Studiengang benotet.

Viel Erfolg!